Unidad 2

JAVASCRIPT II

Definir funciones utilizando =>

Podemos definir funciones sencillas utilizando el operador flecha (=>).

Por ejemplo:

- let multiplicar = (a,b) => a*b;

De esta forma podemos definir una función con el nombre multiplicar y dos argumentos, a y b.

DOM (segunda parte)

Como ya vimos en la unidad anterior, a través de DOM podemos acceder a elementos HTML desde JavaScript, los elementos mas usados de la unidad anterior son:

- document.getElementById(id)
- document.getElementsByTagName(*name*)
- document.getElementById(id).innerHTML
- document.getElementById(id).value
- nodo.textContent

Recordemos que cuando llamamos a un elemento HTML mediante DOM podemos acceder a sus diferentes atributos, como src en un elemento img, o un style.

DOM

A continuación vamos a continuar con otras funciones pertenecientes a DOM:

- document.createElement(“tipo de elemento HTML”): crea un elemento HTML
- document.removeChild(nodo): elimina el nodo hijo del documento
- document.appendChild(nodo) o nodo1.appendChild(nodo2) : añade el nodo al documento
- document. replaceChild(nodo_nuevo, nodo_viejo): o nodo.replaceChild(nodo_nuevo, nodo_viejo): reemplaza un nodo por otro.

Añadir eventos

Podemos añadir eventos a elemento creado mediante DOM:

Nodo.addEventListener(“Evento”, funcion)

```
deleteButton.addEventListener('click', () => {
    fila.remove();
});
```

CUIDADO, en este caso estamos eliminando un nodo.

Callback

Las Callback son funciones que se pasan a otra función por argumento.

Sin embargo las Callback son síncronas, a menudo se utilizan para continuar con la ejecución del código después de que se haya completado una operación asincrónica

```
<body>
    <h2>Ejemplo llamada callback</h2>
    <input type="text" placeholder="Introduce un nombre: " id="name">
    <button onclick="llamada(saludar)">Pulsame</button>

    <script>
        function saludar(nombre){
            alert("Hola " + nombre);
        }

        function llamada(callback){
            let nombre=document.getElementById("name").value;
            callback(nombre);
        }

    </script>
</body>
```

Callback

Esta técnica permite llamar a una función dentro de otra pero sin saber que función se va a utilizar inicialmente.

Las funciones Callback sirven para controlar el orden de ejecución de las distintas funciones según queramos, pudiendo generar funciones asíncronas dependiendo de como las usemos.

```
<button onclick="llamada(saludar)">Pulsame</button>

<!-- <button onclick="saludar(llamada2())">Pulsame</button> -->
```

Promesas

Una promesa es la finalización o fracaso de una tarea asíncrona. Las promesas son en esencia un objeto al cual se adjuntan una serie de funciones callback.

Es importante decir que las promesas solo reciben dos funciones, la función de éxito (resolve) o fracaso (reject);

El comportamiento de estas funciones se definen **dentro** de la promesa.

Definimos una promesa como una función que recibe dos callbacks que luego definimos su comportamiento.

```
<body>
    <h2>Promesas</h2>
    <input type="text" id="ent1" placeholder="numero 1">
    <button onclick="numeroMayor()">Sumar</button>

    <script>
        function alertas(cadena){
            alert(cadena);
        }

        function numeroMayor(){
            let num1=document.getElementById("ent1");

            let promesa = new Promise(function (exito, fracaso){
                if(num1.value>5){
                    exito("El numero es mayor que 5");
                }else{
                    fracaso("El numero no es mayor que 5");
                }
            }
            )

            promesa.then(
                function(value){alertas(value);},
                function(error){alertas(error);}
            )
        }

    </script>
</body>
```

Promesas

Definimos una promesa con dos llamadas callback función(éxito, fracaso), cuando ya hemos definido nuestra promesa, declaramos las funciones de éxito y fracaso

Promesa.then(

Funtion(valor){ secuencia de éxito } ,

Funtion(valor){ secuencia de fracaso}

)

IMPORTANTE: la "," es lo que nos indica que función estamos declarando, a la izquierda de la coma tenemos la función de éxito, a la derecha cuando falla.

Es importante que las funciones que definamos dentro de las promesas no lleven nombre, aunque en sus sentencias podemos usar funciones ya declaradas en otros sitios del documento.

Async/Await

Estos métodos definen una función asíncrona, las funciones async devuelven una promesa, cuando la función devuelva el valor, la promesa utilizara el valor devuelto.

Las funciones async y await tienen la finalidad de simplificar el comportamiento asíncrono de las promesas

La función await se usa en las declaraciones, indica que hasta que no se recibe una promesa, no se sigue con la ejecución normal.

Hay que tener cuidado, ya que no todos los sistemas ni **navegadores** lo **admite**, estas funciones no están disponible en IOS ni el navegador Safari, ni tampoco funciona en Node.js.

```html
<body>

    <script>
        function espera2seg() {
        return new Promise((resolve) => {
            setTimeout(() => {
                resolve("El resultado se muestra despues de que se ejecute esta función y no antes");
            }, 2000);
        });
        }

        async function asyncCall() {
            console.log("Se llama a la función");
            const result = await espera2seg();
            console.log(result);
            console.log("Se muestra despues de la función");
        }

        asyncCall();
    </script>
</body>
```

Recordemos

Async hace que una función **DEVUELVA** una promesa.

Await hace que una función espere hasta que **RECIVA** una promesa.

Una función async puede contener una expresión await, la cual pausa la ejecución de la función asíncrona y espera la resolución de la Promise pasada y, a continuación, reanuda la ejecución de la función async y devuelve el valor resuelto.

Es un método simplificado para utilizar la programación asíncrona.

AJAX

AJAX no es un lenguaje de programación, sino una técnica que nos permite enviar y recibir información de un servidor en segundo plano mediante llamadas asíncronas.

AJAX también es el estándar de programación asíncrona para JavaScript y XML.

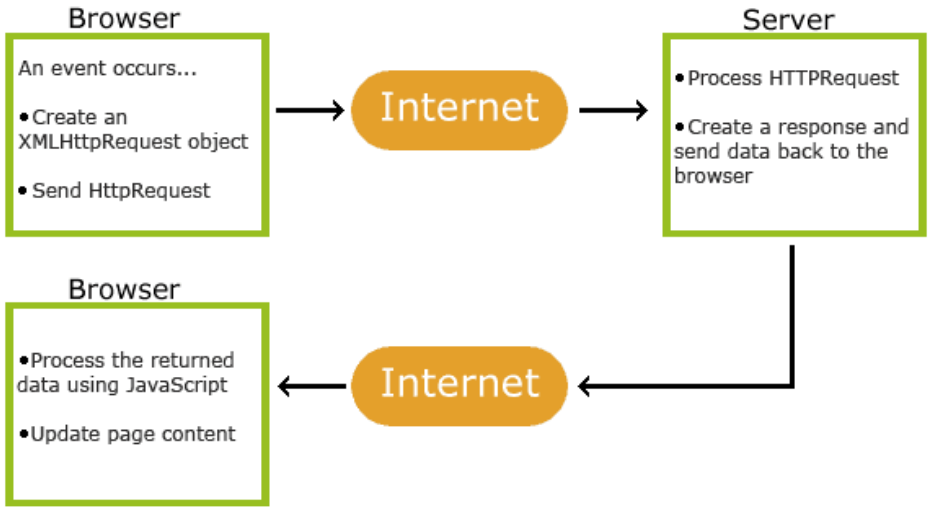
```html
<!DOCTYPE html>
<html>
    <body>
        <h1>Prosema de JavaScript con AJAX</h1>
        <h2>Utilizamos una promesa normal de JAvaScript con then()</h2>

        <p id="demo"></p>

        <script>
            function myDisplayer(some) {
                document.getElementById("demo").innerHTML = some;
            }

            let myPromise = new Promise(function(myResolve, myReject) {
                let req = new XMLHttpRequest();
                req.open('GET', "hola.html");
                req.onload = function() {
                    if (req.status == 200) {
                    myResolve(req.response);
                    } else {
                    myReject("No se encuentra el fichero");
                    }
                };
                req.send();
            });

            myPromise.then(
                function(value) {myDisplayer(value);},
                function(error) {myDisplayer(error);}
            );
        </script>

    </body>
</html>
```

¿Cómo funciona?

Un evento crea una petición http (o cualquier llamada de comunicación al servidor) y envía una petición de intercambio de datos.

El servidor procesa dicha petición y devuelve la información pedida por el navegador.

En el cliente, el navegador recibe la información y modifica la pagina mediante DOM para mostrar el contenido devuelto.



Funciones esenciales

Variable = new XMLHttpRequest(): creamos un objeto petición Http

Variable .open(): mediante un método (GEP, POST…) hacemos una llamada al servidor para que nos devuelva un recurso.

Variable.onload(): definimos una función **Callback**

Variable.send(): enviamos la petición.

Variable.response: la respuesta devuelta por el servidor.

Importante

La función open tiene 5 parámetros open(*método, url_fichero, async/sync, usuario, contraseña*)

- *método: método de comunicación*
- *url_fichero: localización del fichero que queremos*
- *async/sync: por DEFECTO la función se ejecuta de forma ASINCRONA, esto se hace para asegurarse que se dispone del contenido antes de continuar la ejecución. Se introduce true o false (async o sync respectivamente).*
- *Usuario: usuario que ejecuta la consunta (no es necesario si no esta bloqueada la comunicación)*
- *Contraseña: contraseña*

Fetch()

También existe otro método para comunicarnos con el servidor, con el método fetch().

Fetch es una tecnología “experimental” que sustituye al uso de XHTLRequest, en esencia ambos realizan las misma labor, comunicarnos con el servidor y enviar y transmitir información.

Cuando utilizamos el método fetch asignamos su respuesta a una variable.

```html
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Fetch API</h1>
<h2>Se utiliza el método fetch()</h2>
<p id="demo">Este texto va a cambiar cuando se ejecute la función</p>

<script>


async function getText(file) {
  let x = await fetch(file);
  let y = await x.text();
  document.getElementById("demo").innerHTML = y;
}

getText("prueba.txt");
</script>

</body>
</html>
```

prueba.txt

```
1   <p>Hola, esta información se ha extraido mediante el método fetch()</p>
```

JavaScript Fetch API

Se utiliza el mÃ©todo fetch()

Hola, esta información se ha extraido mediante el método fetch()

JSON

¿Qué es JSON? ¿Para que se utiliza?

JSON

JSON es un estándar para la notación de objetos en JavaScript, es un formato de texto plano con una sintaxis similar a los objetos de JavaScript (objetos literales).

```json
{
  "squadName": "Super hero squad",
  "homeTown": "Metro City",
  "formed": 2016,
  "secretBase": "Super tower",
  "active": true,
  "members": [
    {
      "name": "Molecule Man",
      "age": 29,
      "secretIdentity": "Dan Jukes",
      "powers": ["Radiation resistance", "Turning tiny", "Radiation blast"]
    },
    {
      "name": "Madame Uppercut",
      "age": 39,
      "secretIdentity": "Jane Wilson",
      "powers": [
        "Million tonne punch",
        "Damage resistance",
        "Superhuman reflexes"
      ]
    },
    {
      "name": "Eternal Flame",
      "age": 1000000,
      "secretIdentity": "Unknown",
      "powers": [
        "Immortality",
        "Heat Immunity",
        "Inferno",
        "Teleportation",
        "Interdimensional travel"
      ]
    }
  ]
}
```

```javascript
let persona1 = {
    edad: 20,
    nombre: "Alvaro",
    apellidos: "Conde Bazan",
    nombre_completo: {nomb: "Alvaro", apll: "Conde Bazan"},

    salidaNombre: function(){
        return "Mi nombre es: " + this.nombre + " " + this.apellidos;
    },

    salidaNombre2: function(){
        return "Mi nombre es: " + this.nombre_completo.nomb + " " + this.nombre_completo.apll;
    }

}
```

JSON

Usamos el formato JSON para recibir o almacenar información.

Lo cierto es que el formato JSON al utilizar texto plano puede ser utilizado por otros lenguajes de programación.

JSON admite los siguientes tipos de variables:

- Cadenas de texto
- Numeros
- Objetos
- Arrays
- Booleanos
- Null

Cuidado

JSON no admite:

- Funciones
- Formatos de fecha (date)
- Tipos no definidos

JSON

Tenemos dos funciones principalmente para trabajar con JSON:

- JSON.parse(datos_json): transforma los datos en formato JSON en un objeto literal con campos únicamente.
- JSON.stringify(datos_Javascript): transforma los tipos de datos de JavaScript en una cadena JSON.

Utilizamos la función parse cuando recibimos información de un fichero o del servidor.

Utilizamos la función stringify cuando queremos enviar la información o almacenarla.

LocalStorage

LocalStorage es un almacenamiento local que guarda el usuario en la cache de su navegador, un sitio web almacena información del usuario en su navegador para reducir el almacenamiento en el servidor. Esta información solo puede pesar 5 MB como máximo.

Es importante añadir que esta información suele ser archivos de configuración o información no sensible o esencial.

Hay que añadir que los datos están vinculados al origen de donde provienen (dominio, puerto y protocolo)

```html
<h2>Ejemplos con localStorage</h2>

<button onclick="clickCounter()" type="button">Crear/Añadir</button>
<button onclick="borrarDatos()" type="button">Borrar datos</button>

<p>Numero de clicks: <span id="demo"></span></p>

<p>Añadir incrementa el contador interno almacenado de forma local.</p>
<p>Al cerrar la pestaña o el navegador la información no se pierde, a no ser que la borremos nosotros mismos</p>

<script>
document.getElementById("demo").innerHTML = localStorage.clickcount;

function clickCounter() {
  if (localStorage.clickcount) {
    localStorage.clickcount = Number(localStorage.clickcount)+1;
  } else {
    localStorage.clickcount = 1;
  }

document.getElementById("demo").innerHTML = localStorage.clickcount;
}

function borrarDatos(){
    delete localStorage.clickcount;
    //localStorage.clear();
    //localStorage.removeItem("clickcount");

    document.getElementById("demo").innerHTML = localStorage.clickcount;
 }
```

LocalStorage

Las principales funciones son:

- setItem(clave, valor) – almacenar un par clave/valor.
- getItem(clave) – obtener el valor por medio de la clave.
- removeItem(clave) – eliminar la clave y su valor.
- clear() – borrar todo.
- key(índice) – obtener la clave de una posición dada.
- length – el número de ítems almacenados.